Lab # 10

Implementation of Views and Control Structures using Query Language

Objective:

Student will learn about to do the following:

- Views in SQL
- Case operator in SQL
- Union operator in SQL

Software Tools:

- MySQL Workbench

Theory:

In SQL, a view is a virtual table based on the result-set of an SQL statement. A view contains rows and columns, just like a real table. The fields in a view are fields from one or more real tables in the database. You can add SQL functions, WHERE, and JOIN statements to a view and present the data as if the data were coming from one single table.

CREATE VIEW Syntax

```
CREATE VIEW view_name
AS
SELECT column1, column2, ...
FROM table_name
WHERE condition;
```

SQL CREATE VIEW Examples

The following SQL creates a view that shows all customers from XX:

Example

```
CREATE VIEW X Customers
AS
SELECT ID, NAME
FROM Customers
WHERE address = "XX";
```

We can query the view above as follows:

Select * from **X Customers**

SQL Dropping a View

A view is deleted with the DROP VIEW command.

SQL DROP VIEW Syntax

DROP VIEW *view_name*;

The SQL UNION Operator:

The UNION operator is used to combine the result-set of two or more SELECT statements.

- Each SELECT statement within UNION must have the same number of columns
- The columns must also have similar data types
- The columns in each SELECT statement must also be in the same order

UNION Syntax

```
SELECT column_name(s) FROM table1
UNION
SELECT column_name(s) FROM table2;
```

UNION ALL Syntax

The UNION operator selects only distinct values by default. To allow duplicate values, use UNION ALL:

```
SELECT column_name(s) FROM table1
UNION ALL
SELECT column_name(s) FROM table2;
```

SQL UNION Example

The following SQL statement returns the cities (only distinct values) from both the "Customers" and the "Suppliers" table:

Example

```
SELECT City FROM Customers
UNION
SELECT City FROM Suppliers
ORDER BY City;
```

If some customers or suppliers have the same city, each city will only be listed once, because UNION selects only distinct values. Use UNION ALL to also select duplicate values.

```
SELECT City FROM Customers
UNION ALL
SELECT City FROM Suppliers
ORDER BY City;
```

The SQL IF Statement

The IF() function returns a value if a condition is TRUE, or another value if a condition is FALSE.

IF Syntax

```
IF(condition, value_if_true, value_if_false)
```

SQL IF Example

```
SELECT IF(500<1000, 5, 10);
```

```
SELECT OrderID, Quantity, IF(Quantity>10, "MORE", "LESS") AS Return_Value
FROM OrderDetails;
```

The SQL CASE Statement

The CASE statement goes through conditions and returns a value when the first condition is met (like an IF-THEN-ELSE statement). So, once a condition is true, it will stop reading and return the result. If no conditions are true, it returns the value in the ELSE clause.

If there is no ELSE part and no conditions are true, it returns NULL.

CASE Syntax

```
SELECT column1, column2,..
CASE
   WHEN condition1 THEN result1
   WHEN condition2 THEN result2
   WHEN conditionN THEN resultN
   ELSE result
END AS alias_name
FROM Table;
```

SQL CASE Examples

The following SQL goes through conditions and returns a value when the first condition is met:

Example

```
SELECT OrderID, Quantity,
CASE
    WHEN Quantity > 30 THEN "The quantity is greater than
30"
    WHEN Quantity = 30 THEN "The quantity is 30"
    ELSE "The quantity is under 30"
END AS QuantityText FROM OrderDetails;
```

Lab Task:

A. Create table of CE_student, CS_student with having information with the following schema: CE_student (stdID, stdName, dbMarks, cnMarks, gpa, PLO_perc)
CS_student (stdID, stdName, dbMarks, gpa)

Insert five tuples in table and write SQL query/subquery for the following questions:

1. Create view for CE having all data in which database systems marks are greater than average marks.
2. Create view for CE having all data in which computer networks marks are less than average marks .
3. Select stdID and stdName of CE_student whose PLO_perc>70 is categorized as 'safe zone', 70>PLO_perc>50 is categorized as 'margin level' and PLO_perc<50 is categorized as 'below margin'.
4. Select stdID and stdName of CS_students whose gpa>3 is categorized as 'good gpa', 3>gpa>2 is categorized as 'average gpa' and gpa<2 is categorized as 'below average'.
5. Select stdID and stdName of CS and CE students using unions where dbMarks>10 with title CS and CE.

Conclusion:

_____
_____
_____